

Examen de práctica: GitHub y Docker en equipo

CU

Nombre

Dura **90 minutos**. Sin apuntes, formularios ni dispositivos electrónicos: sólo estas hojas, pluma o lápiz y goma. Contesta en el espacio debajo de cada pregunta.

En los parciales, una sola persona cambiaba un archivo y seguías qué pasaba. Aquí trabajan **tres a la vez**: Ana, Beto y Caro. Cada quien tiene su clon, su máquina, sus imágenes, sus contenedores y sus volúmenes. Los cambios llegan de una máquina a otra **sólo** a través de GitHub, con push y pull.

Los dos escenarios se reprodujeron completos: un repositorio local hizo de GitHub, hubo tres clones, y cada comando corrió con Git 2.34 y Docker 29.6. Las salidas que ves son las reales.

Cómo leerlos. Cada fila dice quién actúa y en qué orden; cada fila parte de lo que dejaron las anteriores. Las preguntas usan sólo comandos que ya viste en los parciales. Al final de cada ejercicio no se piden comandos: se pregunta **qué práctica** lo habría evitado.

Una diferencia con el curso. Aquí el equipo comparte **un solo repositorio** donde los tres pueden escribir, como en un proyecto de trabajo. En el curso cada quien tiene su fork y propone por pull request. Con un repositorio compartido aparece algo que con forks casi no pasa: dos personas empujando a la misma rama.

Ejercicio 1 · El umbral

El punto de partida

El equipo mantiene alertas/, un servicio que imprime el umbral con el que filtra. En GitHub, main tiene un solo commit, **B**:

```
alertas/  
├── Dockerfile  
├── alerta.sh  
└── config.env
```

config.env:

```
UMBRAL=5
```

alerta.sh carga ese archivo e imprime el valor:

```
. ./config.env  
echo "umbral: $UMBRAL"
```

Dockerfile: la configuración se **copia dentro de la imagen**.

```
FROM alpine:3.20  
WORKDIR /app  
COPY config.env alerta.sh ./  
CMD ["sh", "alerta.sh"]
```

Los tres ya clonaron el repositorio y, cada uno en su máquina, corrieron:

```
docker build -t alertas:1 .
docker run -d --name alerta alertas:1 sleep 600
```

Así que cada quien tiene una imagen `alertas:1` y un contenedor `alerta` encendido, los dos con `UMBRAL=5`. Ningún `sleep 600` termina durante el ejercicio.

La secuencia

Fila 1 · Ana cambia y sube

```
# Ana
echo "UMBRAL=10" > config.env
git commit -am "umbral 10"
git push origin main
docker run --rm alertas:1
```

¿Qué `UMBRAL` hay ahora en GitHub, en el clon de Beto y en la imagen de Ana? ¿Qué imprime el `docker run` de Ana?

Fila 2 · Ana reconstruye

```
# Ana
docker build -t alertas:1 .
docker run --rm alertas:1
docker exec alerta sh alerta.sh
```

¿Qué imprime el `run`? ¿Y el `exec`?

Fila 3 · Beto, sin haber hecho pull

```
# Beto
echo "UMBRAL=20" > config.env
git commit -am "umbral 20"
git push origin main
```

Git le contesta:

```
! [rejected]        main -> main (fetch first)
hint: Updates were rejected because the remote contains work that you do
```

¿Por qué lo rechaza? ¿Qué UMBRAL hay en GitHub? Si ahora Beto corre `docker build -t alertas:1 .` y `docker run --rm alertas:1`, ¿qué imprime?

Fila 4 · Caro se pone al día

```
# Caro
git pull origin main
docker run --rm alertas:1
docker run --rm -v "$(pwd)/config.env":/app/config.env alertas:1
```

¿Qué hace el pull? ¿Qué imprime cada run?

Fila 5 · Beto integra

```
# Beto
git pull origin main
```

Git contesta:

```
CONFLICT (content): Merge conflict in config.env
Automatic merge failed; fix conflicts and then commit the result.
```

¿Cómo se ve ahora config.env en el disco de Beto? ¿Qué imprime docker exec alerta sh alerta.sh? ¿Y docker run --rm alertas:1?

Fila 6 · Beto «termina»

```
# Beto
git add config.env
git commit -m "listo"
docker build -t alertas:1 .
docker run --rm alertas:1
echo $?
git push origin main
```

Beto no editó el archivo. ¿Git acepta el commit? ¿El build termina bien? ¿Qué imprime el run y qué da echo \$? ¿GitHub acepta el push?

Fila 7 · Caro vuelve a ponerse al día

```
# Caro
git pull origin main
docker run --rm alertas:1
docker run --rm -v "$(pwd)/config.env":/app/config.env alertas:1
```

¿Qué hace el pull? ¿Qué imprime cada run?

Fila 8 · Ana lo arregla

El equipo acuerda un umbral de 15.

```
# Ana
git pull origin main
echo "UMBRAL=15" > config.env
git commit -am "umbral acordado 15"
git push origin main
docker build -t alertas:1 .
docker run --rm alertas:1
```

¿Cómo sale el pull? ¿Qué imprime el run?

Fila 9 · La foto final

Nadie más hace nada. Llena la tabla con lo que dice cada lugar: un número, «marcadores» o «trueno».

Lugar	¿Qué dice?
main en GitHub	
Disco de Ana	
Disco de Beto	
Disco de Caro	
Imagen de Ana	
Imagen de Beto	
Imagen de Caro	
Contenedor alerta de Ana	
Contenedor alerta de Beto	
Contenedor alerta de Caro	

Diagnóstico y prácticas

Aquí no se piden comandos: se pide entender qué falló y qué costumbre del equipo lo habría evitado.

D1 • ¿Dónde entró el error?

¿En qué fila entró el archivo roto a main? ¿Quién pudo detenerlo antes, y con qué señal?

D2 • ¿Por qué cada quien ve otra cosa?

Al final hay cuatro valores distintos de UMBRAL en el equipo. Explica las **tres** razones por las que una persona puede estar corriendo una versión distinta de la que hay en GitHub.

D3 · Las prácticas que faltaron

Sin escribir comandos: ¿qué cuatro o cinco costumbres de equipo habrían evitado este ejercicio completo? Para cada una, di qué fila habría cambiado.

Ejercicio 2 · La base de datos

El punto de partida

El equipo mantiene tienda/, con una base Postgres. Los cambios a la base son archivos SQL versionados en Git:

```
tienda/  
└─ sql/  
   └─ 001_clientes.sql
```

001_clientes.sql:

```
CREATE TABLE clientes (id serial PRIMARY KEY, nombre text);
```

Cada quien levanta la base en su máquina, desde la carpeta del repo, siempre con el mismo comando:

```
docker run -d --name db -e POSTGRES_PASSWORD=fdd \  
-v datos:/var/lib/postgresql/data \  
-v "$(pwd)/sql":/docker-entrypoint-initdb.d \  
postgres:16
```

- datos es un **named volume**: ahí vive la base, y sobrevive a `docker rm`.
- El segundo `-v` monta la carpeta `sql/` del repo donde la imagen busca scripts de arranque.

Un dato que necesitas. Al arrancar, la imagen de Postgres:

- corre los `.sql` de `/docker-entrypoint-initdb.d`, en orden alfabético, **sólo si el volumen de datos está vacío**;
- si el volumen ya tiene una base, se salta todos los scripts y arranca;
- si un script falla, se detiene, y el contenedor se apaga.

Para ver las tablas: `docker exec db psql -U postgres -c '\dt'`.

Los tres clonaron el repo y levantaron su base. Cada quien tiene un volumen datos con la tabla clientes.

La secuencia

Fila 1 · Ana agrega la tabla ventas, en una rama

```
# Ana
git switch -c ventas
# crea sql/002_ventas.sql:
# CREATE TABLE ventas (id serial PRIMARY KEY,
#                           cliente int REFERENCES clientes(id), total numeric);
git add sql
git commit -m "tabla ventas"
git push -u origin ventas
docker rm -f db
docker run -d --name db ... postgres:16      # el mismo comando de arriba
docker exec db psql -U postgres -c '\dt'
```

¿Qué tablas ve Ana?

Fila 2 · Ana prueba con un volumen nuevo

```
# Ana
docker rm -f db
docker volume rm datos
docker run -d --name db ... postgres:16
docker exec db psql -U postgres -c '\dt'
```

¿Qué tablas ve ahora? ¿Qué perdió?

Fila 3 · Beto agrega reportes, en su rama

Al mismo tiempo, Beto necesita una tabla con ventas por mes:

```
# Beto
git switch -c reportes
# crea sql/003_ventas_mensuales.sql:
# CREATE TABLE ventas (mes text PRIMARY KEY, total numeric);
git add sql
git commit -m "ventas mensuales"
git push -u origin reportes
docker rm -f db
docker volume rm datos
docker run -d --name db ... postgres:16
docker exec db psql -U postgres -c '\dt'
```

¿Qué tablas ve Beto?

Fila 4 · Se mergean los dos pull requests

En GitHub alguien mergea el #1 y luego el #2. ¿Hay conflicto? ¿Por qué? ¿Qué archivos hay ahora en sql/ en main?

Fila 5 · Ana y Beto se ponen al día

Cada uno, en su máquina, con su volumen de la fila 2 y de la fila 3:

```
# Ana, y luego Beto
git switch main
git pull origin main
docker rm -f db
docker run -d --name db ... postgres:16
docker exec db psql -U postgres -c '\d ventas'
```

\d ventas muestra las columnas de la tabla. ¿Qué columnas ve cada uno? ¿Corrió algún script?

Fila 6 · Caro se pone al día

Caro todavía tiene su volumen del principio.

```
# Caro
git pull origin main
docker rm -f db
docker run -d --name db ... postgres:16
docker exec db psql -U postgres -c '\dt'
```

¿Qué tablas ve?

Fila 7 · Caro empieza desde cero

```
# Caro
docker rm -f db
docker volume rm datos
docker run -d --name db ... postgres:16
docker ps
docker logs db
```

¿Aparece db en docker ps? ¿Qué dice el log?

Fila 8 · Caro lo intenta otra vez

```
# Caro
docker rm -f db
docker run -d --name db ... postgres:16
docker exec db psql -U postgres -c '\dt'
```

No borró el volumen. ¿Arranca? ¿Qué tablas ve? ¿Por qué esto es **peor** que el error de la fila 7?

Fila 9 · La contraseña

Beto sube por error un archivo con la contraseña real de producción, y en el siguiente commit lo borra:

```
# Beto
echo "POSTGRES_PASSWORD=Tienda2026!" > .env
git add .env
git commit -m "config local"
git push origin main
rm .env
git commit -am "quita .env"
git push origin main
```

Alguien clona el repo después. ¿Ve .env en su disco? ¿La contraseña sigue en GitHub?

Diagnóstico y prácticas

D1 · ¿Qué tiene cada base?

Después de la fila 8, ¿qué tablas y qué columnas de ventas tiene la base de Ana, la de Beto y la de Caro? ¿Cuál de las tres es «la correcta»?

D2 · Sin conflicto, pero roto

Git dijo «sin conflicto» en la fila 4. Explica por qué eso no garantizaba nada.

D3 · Las prácticas que faltaron

Sin escribir comandos: ¿qué costumbres habrían evitado los problemas de las filas 1, 5, 7, 8 y 9?
